BSc (Hons) in **Information Technology Specialized in AI**

Faculty of Computing

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Practical 01

Objective & Lecture Mapping: Recall that an intelligent agent acts within a continuous loop: Sensors generate percept sequences, and Actuators execute actions to maximize a Performance measure. Use this sheet to execute practical modifications and incrementally evaluate your design against Lecture 01 and 02 theory (from basic recall to analytical classification).

Part 1: Code Analysis & PEAS Evaluation

Task 0 : Setting up and getting the starter Code

1. Go to the Git Hub Repo. Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" brach of the repo.

Task 1.1: The Environment State (__init__)

1. (Remember) List the four components of the PEAS framework discussed in Lecture 01.

Performance measure - the metric used to judge success (e.g., score)
Environment - the world the agent operates in (grid, walls, food, opponents)
Actuators - how the agent acts on the world (move up/down/left/right)
Sensors - how the agent perceives the world (percept dictionary)

2. (Understand) Look at the `VisualGridHuntGame.__init__` method. Which specific variables in this code represent the physical "Environment (E)" state?

self.width & self.height (grid dimensions), self.walls (wall coordinates), self.food_positions (food coordinates), self.opponents (opponent positions)

3. (Analyze) Based on the variables initialized (specifically `self.opponents`), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.

Multi-Agent - self.opponents is non-empty and those opponents act (move, hunt, etc.) independently

Task 1.2: The Perception Subsystem (get_percept)

4. (Remember) Define what a "percept sequence" is according to Lecture 01.

The complete history of every percept the agent has received so far, in order, not just the current percept

5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?

Sensors - It's the function that packages the observable world state into the percept dict passed to the agent

6. (Evaluate) Based on the exact dictionary returned by `get_percept()`, is this environment "Fully Observable" or "Partially Observable"? Explain why based on what the agent can and cannot see.

Partially Observable - "remaining_food" only tells the agent the remaining number of food and "smells_food" only tells the agent if it's standing on food, and neither tells the exact position of food in the grid. Hence it's a Partially Observable environment

Task 1.3: Action Execution (execute_action)

7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?

Performance measure - points are actively deducted when the agent hits a wall, which is a Performance measure

8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

Measuring external results keeps the metric objective and comparable across agents - it judges outcomes anyone can verify by observing the environment, not the agent's hidden internal reasoning. If effort were rewarded instead, two agents with identical real-world outcomes could get different scores, and the metric would become subjective and easy to game (e.g., "trying hard" without actually avoiding walls).

Part 2: Guided Modification & Deep Theory Mapping

> Follow the implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical nature of the environment.

Step 2.1: Extending the Environment Initialization (__init__)

1. **Implementation Instructions:** Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `__init__` . Populate it with coordinate tuples safely avoiding position `(0, 0)` , walls, and food.

9. (Understand) If you successfully add `self.toxic_traps` to the environment but intentionally hide this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?

Currently the environment is already partially observable (food/wall layout is hidden). Adding toxic_traps to the true state while not exposing it through get_percept() makes the environment even more partially observable - there's now an additional hazardous feature of the world that the agent has zero sensory access to, widening the gap between the true state and what the agent perceives.

Step 2.2: Updating the Perception Subsystem (get_percept)

1. **Implementation Instructions:** Locate `get_percept(self)` and add a new boolean sensor key (e.g., `'smells_toxin': tuple(self.agent_pos) in self.toxic_traps`) to the dictionary returned to the agent loop.

10. (Analyze) By adding `'smells_toxin'`, you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing expected utility).

Without this key, the agent has no percept-based way to anticipate a trap — it would only find out after already stepping into one and losing points, which is too late to act rationally. With smells_toxin in the percept, the agent gains information it can use before choosing its next action — e.g., avoid repeating a move that led to a smells_toxin: True percept, or treat it as negative feedback to steer away from that cell. This lets a rational agent select actions that are expected to maximize its score (avoid penalties) using only the information available to it.

Step 2.3: Modifying Action Execution & Visual Rendering (execute_action & draw_grid)

1. **Implementation Instructions:** In `execute_action`, check if the updated `self.agent_pos` intersects with `self.toxic_traps` to decrement `self.score` by 15 points. In `draw_grid`, iterate through traps to render custom purple shapes on the canvas.

11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example discussed in Lecture 01 that illustrated the danger of faulty metrics?

In the original vacuum-world example, if the performance measure rewards "amount of dirt cleaned per time step" rather than "floor is clean," a rational agent can exploit this by repeatedly dumping dirt back onto a clean square and then cleaning it again, racking up reward indefinitely without ever accomplishing the actual goal. It's the textbook illustration of why a poorly designed metric lets an agent game the proxy instead of achieving the real objective - which is why this trap penalty is tied to an actual external event (touching a trap) rather than something the agent could manipulate for free score.

Finalize and Lock Lab 01 Analytical Evaluation

FACULTY OF COMPUTING